第20章源码阅读

由正式源文件生成的静态阅读副本。返回正文 · 运行说明

---

src/main/java/org/example/jvminternals/ch20/SafepointJourney.java

```
package org.example.jvminternals.ch20;

import java.util.concurrent.CountDownLatch;

/** 在正常线程协作中关联分配阶段、安全点与回收操作。 */
public final class SafepointJourney {
    private static final System.Logger LOG =
System.getLogger(SafepointJourney.class.getName());
    private static volatile byte[] latest;

    private SafepointJourney() { }

    /** 主线程只请求两次回收，等待教学线程自然完成。 */
    public static void main(String[] args) throws InterruptedException {
        CountDownLatch started = new CountDownLatch(1);
        Thread worker = new Thread(() -> allocate(started), "分配教学线程");
        worker.start();
        started.await();
        LOG.log(System.Logger.Level.INFO, "阶段=主线程第一次请求回收");
        System.gc();
        Thread.sleep(200);
        LOG.log(System.Logger.Level.INFO, "阶段=主线程第二次请求回收");
        System.gc();
        worker.join();
        LOG.log(System.Logger.Level.INFO, "阶段=线程收尾，末块字节={0}", latest[0]);
    }

    /** 固定24批；休眠用于分开展示窗口，不作为正确性判定条件。 */
    private static void allocate(CountDownLatch started) {
        LOG.log(System.Logger.Level.INFO, "阶段=教学线程启动");
        started.countDown();
        try {
            for (int phase = 0; phase < 24; phase++) {
                for (int item = 0; item < 64; item++) {
                    byte[] payload = new byte[8192];
                    payload[0] = (byte) phase;
                    latest = payload;
                }
```

```
                Thread.sleep(25);
            }
            LOG.log(System.Logger.Level.INFO, "阶段=教学线程自然完成");
        } catch (InterruptedException interrupted) {
            Thread.currentThread().interrupt();
            LOG.log(System.Logger.Level.WARNING, "教学线程被中断，提前结束分配", interrupted);
        }
    }
}
```